

CI/CD Pipeline with CodePipeline, CodeBuild, EC2, PM2, SSM & Blue/Green Deployment

Project Overview

This project demonstrates a complete CI/CD (Continuous Integration and Continuous Deployment) pipeline on AWS using:

- GitHub
- AWS CodePipeline
- AWS CodeBuild
- AWS Systems Manager
- Amazon EC2
- Nginx
- PM2

The project automatically:

- pulls code from GitHub
- builds and tests application
- deploys latest code to EC2
- restarts application automatically
- supports Blue/Green deployment

This documentation style follows the uploaded scalable AWS deployment document structure.

Definition

CI/CD Pipeline is an automated software delivery process where code changes are automatically built, tested, and deployed to servers without manual intervention.

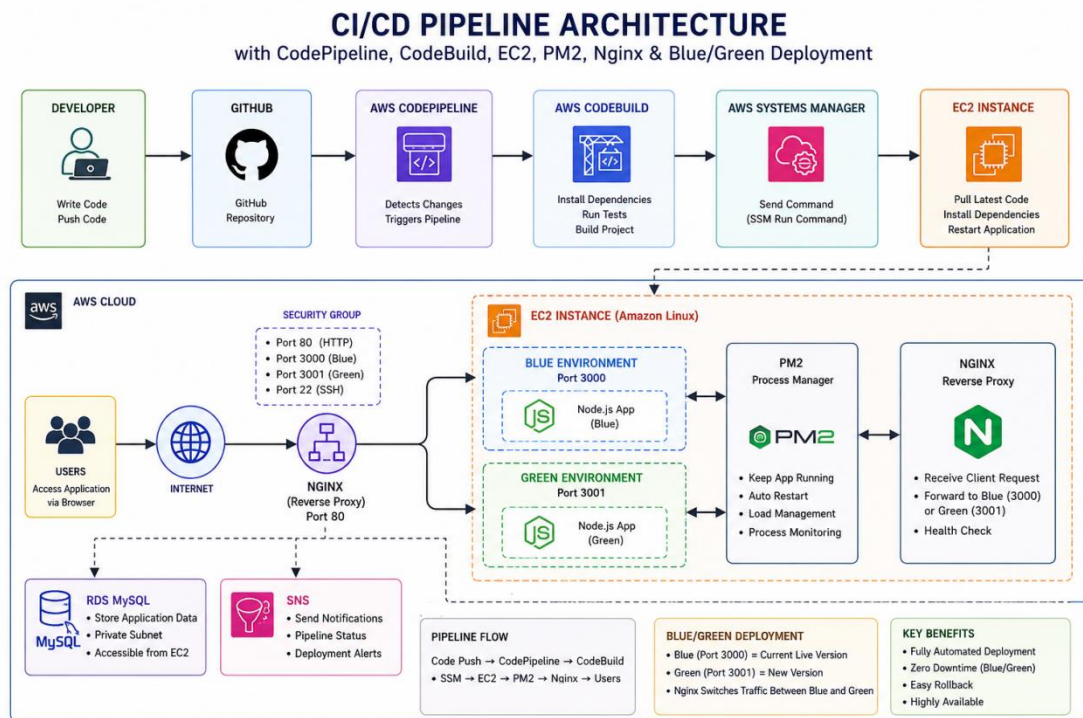
- CI = Continuous Integration
- CD = Continuous Deployment

Goal of the Project

The main goal of this project is:

- automate application deployment
- reduce manual work
- test code automatically
- deploy latest code automatically
- implement Blue/Green deployment strategy

Architecture



AWS Services Used in the Project

1. GitHub

Definition

GitHub is a cloud-based platform used to store and manage source code using Git version control.

Purpose

Used to store Node.js application code.

Role in Project

Whenever new code is pushed to GitHub, CodePipeline automatically detects changes and starts the CI/CD workflow.

2. AWS CodePipeline

Definition

AWS CodePipeline is a CI/CD orchestration service that automates software release workflows.

Purpose

Used to automate the complete pipeline flow.

Role in Project

CodePipeline:

- detects GitHub code changes
- triggers CodeBuild
- controls deployment workflow

3. AWS CodeBuild

Definition

AWS CodeBuild is a fully managed build service used to compile code, install dependencies, and run tests.

Purpose

Automatically build and test the Node.js application.

Role in Project

CodeBuild:

- downloads source code
- installs npm packages
- runs tests
- triggers deployment commands

4. AWS Systems Manager (SSM)

Definition

AWS Systems Manager is a management service used to remotely manage AWS resources securely.

Purpose

Used to run deployment commands on EC2 without SSH keys.

Role in Project

SSM allows CodeBuild to:

- connect EC2 securely
- run deployment commands
- restart application automatically

5. Amazon EC2

Definition

Amazon EC2 provides virtual Linux servers in AWS cloud.

Purpose

Used to host the Node.js application.

Role in Project

EC2 server:

- runs backend application
- stores project files
- executes deployment updates

6. PM2

Definition

PM2 is a Node.js process manager used to run applications in the background.

Purpose

Keeps Node.js application continuously running.

Role in Project

PM2:

- runs app in background
- auto restarts crashed apps
- survives server reboot
- manages Blue and Green applications

7. Nginx Reverse Proxy

Definition

Nginx is a web server and reverse proxy server that forwards client requests to backend applications.

Purpose

Used to route traffic to Node.js application.

Role in Project

Nginx:

- receives traffic on port 80
- forwards traffic to Node.js app
- switches traffic between Blue and Green environments

8. IAM Roles

Definition

IAM roles provide permissions for AWS services to interact securely.

Purpose

Allow AWS services to perform required actions.

Role in Project

IAM roles were used for:

- CodeBuild permissions
- SSM permissions
- GitHub connection permissions
- EC2 Systems Manager acce

Project Workflow

```

Developer Pushes Code
↓
GitHub Repository Updated
↓
CodePipeline Detects Changes
↓
CodeBuild Starts Automatically
↓
npm install
npm test
↓
SSM Deployment Command
↓
EC2 Executes Deployment
↓
PM2 Restarts Application
↓
Nginx Serves Latest Version

```

Project Folder Structure

```

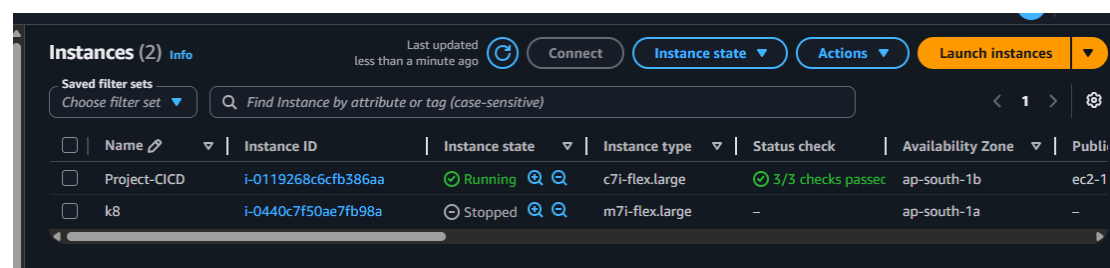
cicd-node-app/
├── app.js
├── package.json
├── package-lock.json
├── .env
├── buildspec.yml
├── routes/
│   └── userRoutes.js

```

Step 1 — Launch EC2 Instance

Created EC2 instance with:

Setting	Value
AMI	Amazon Linux 2023
Type	t2.micro
Storage	8 GB
OS	Amazon Linux



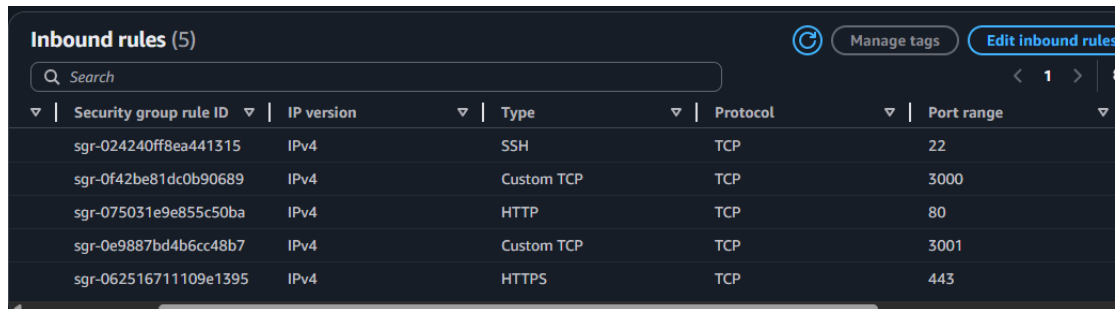
Purpose

EC2 server is required to host and run the Node.js application.

Step 2 — Configure Security Group

Created security group:

cicd-sg



The screenshot shows the 'Inbound rules (5)' section of the AWS IAM console. It displays a table with 5 rules. The table has columns for Security group rule ID, IP version, Type, Protocol, and Port range. The rules are:

Security group rule ID	IP version	Type	Protocol	Port range
sgr-024240ff8ea441315	IPv4	SSH	TCP	22
sgr-0f42be81dc0b90689	IPv4	Custom TCP	TCP	3000
sgr-075031e9e855c50ba	IPv4	HTTP	TCP	80
sgr-0e9887bd4b6cc48b7	IPv4	Custom TCP	TCP	3001
sgr-062516711109e1395	IPv4	HTTPS	TCP	443

Purpose

- Port 80 → Nginx
- Port 3000 → Blue app
- Port 3001 → Green app

Step 3 — Install Node.js and Git

Installed packages:

```
sudo dnf update -y
sudo dnf install nodejs -y
sudo dnf install git -y
```

Purpose

Required for:

- running Node.js application
- cloning GitHub repository
- managing npm packages

Step 4 — Create Node.js Application

Installed Packages

```
npm install express cors dotenv
npm install --save-dev nodemon
```

Step 5 — Create app.js

```
const express = require("express");
const cors = require("cors");
require("dotenv").config();

const userRoutes = require("./routes/userRoutes");

const app = express();

app.use(cors());
app.use(express.json());

app.get("/", (req, res) => {
  res.send("CI/CD Pipeline Working");
});

app.get("/health", (req, res) => {
  res.json({
    status: "OK"
  });
});

app.use("/api/users", userRoutes);

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 6 — Create Environment File

Created:

PORT=3000

Purpose

Environment variables help separate configuration from code.

Step 7 — Configure package.json Scripts

```
"scripts": {  
  "start": "node app.js",  
  "dev": "nodemon app.js",  
  "test": "echo \"Tests passed successfully\" && exit 0"  
}
```

Purpose

Script	Purpose
start	Run production app
dev	Development mode
test	Used by CodeBuild

```
[root@ip-172-31-1-205 cisd-node-app]# ls  
app.js  buildspec.yml  node_modules  package-lock.json  package.json  routes  
[root@ip-172-31-1-205 cisd-node-app]#
```

Step 8 — Push Project to GitHub

Commands used:

```
git init  
git add .  
git commit -m "initial commit"  
git branch -M main  
git push -u origin main
```

Purpose

Upload project to GitHub repository.

root retry deploy 45d8c96 · 3 hours ago 7 Commits		
node_modules	initial commit	5 hours ago
routes	initial commit	5 hours ago
.env	initial commit	5 hours ago
app.js	test pipeline	3 hours ago
buildspec.yml	fix ccm deploy command	3 hours ago

Step 9 — Create buildspec.yml

Definition

buildspec.yml is a YAML file used by CodeBuild to know what commands to execute.

buildspec.yml

```
version: 0.2
```

```
phases:
```

```
  install:
```

```
    runtime-versions:
```

```
      nodejs: 20
```

```
    commands:
```

```
      - echo Installing dependencies
```

```
      - npm install
```

```
  pre_build:
```

```
    commands:
```

```
      - echo Running tests
```

```
      - npm test
```

```
  build:
```

```
    commands:
```

```
      - echo Build successful
```

```
  post_build:
```

```
    commands:
```

```
      - echo Deploying to EC2 via SSM
```

```
      - aws ssm send-command --instance-ids "i-0119268c6cfb386aa" --document-name  
"AWS-RunShellScript" --comment "Deploy Node App" --parameters 'commands=["cd  
/home/ec2-user/cicd-node-app && git pull origin main && npm install && pm2 restart cicd-  
app"]'
```

```
artifacts:
```

```
  files:
```

```
    - '**/*'
```

Purpose

This file tells CodeBuild:

- what to install
- what tests to run
- how deployment should happen

Step 10 — Create IAM Role for CodeBuild

Created role:

codebuild-cicd-role

Attached policies:

AWSCodeBuildDeveloperAccess

CloudWatchLogsFullAccess

AmazonS3FullAccess

AmazonSSMFullAccess

☐	Policy name ↗	Type	Attached entities
☐	  AmazonS3FullAccess	AWS managed	1
☐	  AmazonSSMFullAccess	AWS managed	1
☐	  AWSCodeBuildDeveloperAccess	AWS managed	1
☐	  CloudWatchFullAccess	AWS managed	1
☐	 CodeBuildBasePolicy-codebuild-ci...	Customer managed	1
☐	 CodeBuildGitHubConnectionAccess	Customer inline	0

Purpose

Allow CodeBuild to:

- access GitHub source
- store logs
- deploy to EC2 using SSM

Step 11 — Create GitHub Connection

Created:

- GitHub App connection
- authorized AWS access

Purpose

Secure integration between:

- GitHub
- CodeBuild
- CodePipeline

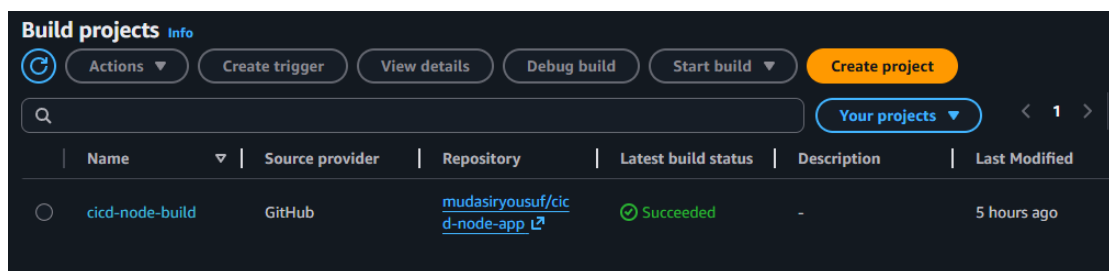
Step 12 — Create CodeBuild Project

Created:

cicd-node-build

Configuration

Setting	Value
Source	GitHub
OS	Amazon Linux
Runtime	Standard
Service Role	codebuild-cicd-role



Purpose

Automatically build and test project.

Step 13 — Create CodePipeline

Created:

cicd-node-pipeline-v2

Stages

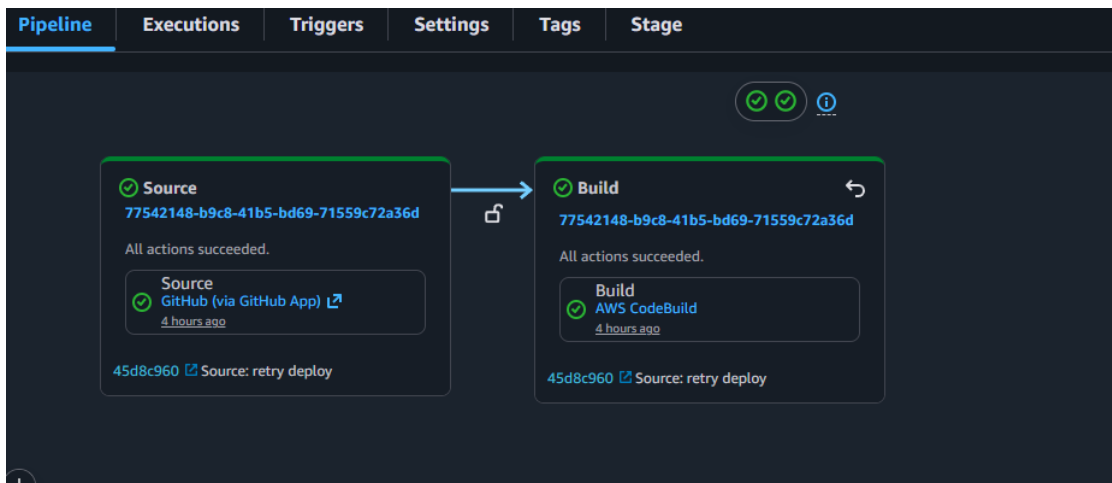
Source Stage

- GitHub repository
- main branch

Build Stage

- AWS CodeBuild

Pipelines Info				
View history Release change Delete pipeline Create pipeline				
Name	Latest execution status	Latest source revisions	Latest execution started	Most recent executions
☐ cicd-node-pipeline-v2	✓ Succeeded	Source – 45d8c960 retry deploy	4 hours ago	✓ ✗ ✗ ✓ ✓ View details



Purpose

Automates:

- build trigger
- deployment workflow

Step 14 — Configure SSM Access for EC2

Created IAM role:

EC2-SSM-Role

Attached policy:

AmazonSSMManagedInstanceCore

Attached role to EC2 instance.

Permissions policies (1) Info				
Simulate Remove Add permissions				
Filter by Type All types				
Policy name	Type	Attached entities		
☐ AmazonSSMManagedInstanceCore	AWS managed	2		

Purpose

Allows AWS Systems Manager to manage EC2 securely.

Step 15 — Install PM2

Installed:

```
sudo npm install -g pm2
```

Started app:

```
pm2 start app.js --name cisd-app
```

Saved process:

pm2 save

```
bash: Starting /home/ec2-user/cisd-node-app-green/app.js in fork_mode (1 instance)
[PM2] Done.
```

id	name	namespace	version	mode	pid	uptime	U	status	cpu	mem	user	watching
0	cisd-app	default	1.0.0	fork	2837	73m	0	online	100%	57.1mb	root	disabled
1	green-app	default	1.0.0	fork	7089	0s	0	online	0%	14.0mb	root	disabled

```
[root@ip-172-31-1-205 cisd-node-app-green]# pm2 list
```

id	name	namespace	version	mode	pid	uptime	U	status	cpu	mem	user	watching
0	cisd-app	default	1.0.0	fork	2837	73m	0	online	0%	57.1mb	root	disabled
1	green-app	default	1.0.0	fork	7089	7s	0	online	0%	55.4mb	root	disabled

```
[root@ip-172-31-1-205 cisd-node-app-green]#
```

Purpose

PM2:

- keeps app running
- auto restarts crashes
- runs app in background

Step 16 — Install Nginx

Installed:

```
sudo dnf install nginx -y
```

Started:

```
sudo systemctl start nginx
sudo systemctl enable nginx
```

```
bash: Started: command not found
[root@ip-172-31-1-205 ec2-user]#
[root@ip-172-31-1-205 ec2-user]# sudo systemctl enable nginx
[root@ip-172-31-1-205 ec2-user]# sudo systemctl status nginx
● nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Tue 2026-05-26 09:22:59 UTC; 3h 16min ago
     Main PID: 7333 (nginx)
        Tasks: 3 (limit: 4523)
       Memory: 3.5M
          CPU: 41ms
     CGroup: /system.slice/nginx.service
```

Step 17 — Configure Reverse Proxy

Edited:

[/etc/nginx/nginx.conf](#)

Added:

```
location / {  
    proxy_pass http://localhost:3000;  
    proxy_http_version 1.1;  
  
    proxy_set_header Upgrade $http_upgrade;  
    proxy_set_header Connection 'upgrade';  
    proxy_set_header Host $host;  
}
```

Restarted Nginx:

```
sudo nginx -t  
sudo systemctl restart nginx
```

Purpose

Users can now access application using:

http://PUBLIC_IP

instead of:

http://PUBLIC_IP:3000

CI/CD Nooode App 

Deployed using AWS CodePipeline + CodeBuild + EC2

[Health Check](#)

Step 18 — Blue/Green Deployment

Definition

Blue/Green deployment is a deployment strategy where two separate environments exist:

- Blue = current live version
- Green = new version

Traffic can switch between them without downtime.

Blue Environment

Item Value

App Blue

Port 3000

Green Environment

Created copy:

```
cp -r cicd-node-app cicd-node-app-green
```

Created .env

PORT=3001

Started Green app:

```
pm2 start app.js --name green-app
```

```
[root@ip-172-31-1-205 ec2-user]# ls
cicd-node-app  cicd-node-app-green
[root@ip-172-31-1-205 ec2-user]#
```

Traffic Switching

Blue:

```
proxy_pass http://localhost:3000;
```

Green:

```
proxy_pass http://localhost:3001;
```

Restart:

```
sudo systemctl restart nginx
```


CI/CD Nooode App

Deployed using AWS CodePipeline + CodeBuild + EC2

[Health Check](#)

Purpose

Switch traffic instantly without downtime.

Automatic Deployment Flow

```
git push
↓
CodePipeline triggers
↓
CodeBuild runs
↓
npm install
npm test
↓
SSM deployment command
↓
EC2 git pull
↓
npm install
↓
pm2 restart
```

Troubleshooting

Issue 1 — GitHub Repo Not Showing

Cause

GitHub connection not configured.

Solution

Created GitHub App connection.

Issue 2 — npm run dev Missing

Cause

Broken package.json.

Solution

Recreated complete JSON file.

Issue 3 — SSM Access Denied

Cause

CodeBuild role lacked SSM permissions.

Solution

Attached:

AmazonSSMFullAccess

Skills Learned

- CI/CD concepts
- GitHub integration
- AWS CodeBuild
- AWS CodePipeline
- AWS Systems Manager
- IAM roles
- PM2
- Nginx reverse proxy
- Blue/Green deployment
- Linux server management
- Automated deployment workflow

Conclusion

This project successfully implements a complete CI/CD pipeline using AWS services. The application automatically builds, tests, and deploys code changes from GitHub to EC2. PM2 ensures high availability while Nginx supports Blue/Green deployment strategy for safer production releases.

The project demonstrates:

- automation
- continuous integration
- continuous deployment
- deployment orchestration
- traffic management
- production-style DevOps workflow

The documentation structure and explanation style are inspired by the uploaded scalable AWS project document.